Time Series Analysis in Financial Markets: A Matrix Profiling and Volatility Modeling Approach

Project Report

IT340M

MACHINE LEARNING

by

Nithin S

Roll.No. 221IT085

Department of Information Technology
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA (NITK)
SURATHKAL, MANGALORE - 575 025
December 1, 2024

DECLARATION

By the Student

I hereby *declare* that the Project entitled **"Time Series Analysis in Financial Markets: A Matrix Profiling and Volatility Modeling Approach"**, which is being submitted to the ***National Institute of Technology Karnataka, Surathkal*** is a *bonafide report of the project work carried out by me*.

Nithin S
Roll No.: 221IT085
Department of Information Technology
National Institute of Technology Karnataka, Surathkal

Place: NITK, Surathkal

Date: December 1, 2024

ABSTRACT

Abstract

Time series analysis is a crucial statistical technique used to analyze data points collected sequentially over time. This project explores various methodologies for understanding, modeling, and predicting time series data. The study begins with an introduction to time series analysis, followed by the extraction and preparation of financial data using the `yfinance` library. Advanced methods, such as STUMPY's matrix profiling for motif discovery and user-guided annotation vector-based motif searches, are implemented and discussed. The project further employs classical time series modeling techniques, including the Autocorrelation Function (ACF), Partial Autocorrelation Function (PACF), ARMA, and GARCH models, to analyze and forecast time-dependent data. Finally, the project concludes with a summary of results and a discussion of potential areas for future research. This comprehensive approach aims to provide a solid foundation for time series analysis and its practical applications in finance and beyond.

Keywords: Time Series Analysis, Stumpy, yfinance, Autocorrelation Function (ACF), Partial Au- tocorrelation Function (PACF), Autoregressive Moving Average (ARMA) , Generalized Autoregressive Conditional Heteroskedasticity (GARCH)

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Time Series Analysis

Time series analysis is a statistical method used to study and interpret patterns in data that unfold over time. If you have ever glanced at stock market charts, those intricate graphs depict a prime example of time series data in action.

Imagine looking at a series of data points collected over time, such as stock prices fluctuating up and down. Machine learning examines this data to identify patterns or trends, like determining whether stock prices tend to rise or fall during specific times of the year.

Forecasting in time series analysis can be likened to using a highly intelligent crystal ball. It utilizes historical data to make educated guesses about future outcomes. Instead of asserting, "This will definitely happen," it suggests, "Based on previous data, here is a reasonable estimate of what might happen next." While not perfect, this approach provides a valuable tool for making predictions.

1.1.1 Key Components of Time Series Analysis

Time Series Data

Time series data consists of observations or measurements collected over time, with each data point corresponding to a specific moment. Examples include stock prices, temperature readings, and sales figures.

Components of a Time Series

- **Trend:** The long-term movement or direction in the data, representing an underlying pattern that may increase, decrease, or remain relatively constant over time.
- **Seasonality:** Repeating patterns or cycles that occur at regular intervals, often tied to seasons, months, days, or specific times of the day.
- **Cyclic Patterns:** Longer-term fluctuations without fixed periods that may not occur regularly.
- **Residual Component:** Irregular or random fluctuations that deviate from trends, seasonal, or cyclical patterns.

Methods for Time Series Analysis

- **Descriptive Statistics:** This involves basic metrics to understand central tendencies and data spread, such as mean, median, and standard deviation.
- **Visualization:** Graphs and charts, such as line plots, reveal patterns and trends over time.
- **Smoothing Methods:** Used to reduce noise and highlight trends, such as moving averages, which show average values over a period.
- **Statistical Models:** Advanced techniques to model and predict trends, including ARIMA (AutoRegressive Integrated Moving Average) and more sophisticated models like LSTM (Long Short-Term Memory) networks, which excel at detecting patterns in sequential data.

Forecasting

Time series analysis often includes predicting future values by examining historical patterns. Forecasting techniques range from basic extrapolation to advanced predictive modeling.

Evaluation

The performance of time series models is evaluated using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

Objectives of the Project

- To introduce the fundamental concepts of time series analysis.
- To extract and prepare financial data for analysis using the `yfinance` library.
- To implement matrix profiling and motif discovery using the STUMPY library.
- To explore user-guided annotation vector-based motif searches for enhanced pattern identification.
- To analyze time series data using classical methods such as ACF, PACF, ARMA, and GARCH.
- To summarize the results and provide insights for future research directions in time series modeling.

1.2 Organisation of the Project

The thesis is organised as follows:

Chapter 1: This chapter gives a brief introduction to Time Series Analysis.

Chapter 2: This chapter gives an idea about the data structure from yfinance,

Chapter 3: This chapter discuss about the STUMPY implementation of matrix profiling and motif discovery.

Chapter 4: This chapter provides the implementation using Autocorrelation Function, PArtial Autocorrelation Function, ARMA, and GARCH.

Chapter 5: The project is summarised here with a discussion on results and future research.

Chapter 2

Data Acquisition and Structure from Yahoo Finance (yfinance)

2.1 Introduction to yfinance

Yahoo Finance (yFinance) is a Python library that simplifies financial data retrieval from the Yahoo Finance platform Aroussi (2019). It provides historical and real-time data for stocks, indices, and other financial instruments. By specifying parameters such as the stock ticker, date range, and interval, users can easily download structured financial data for analysis and visualization.

2.2 Data Retrieval Methodology

For this project, we used yfinance to fetch minute-wise stock data for the Nifty 50 index over the last 30 days. As yfinance imposes a restriction of providing a maximum of 7 days of minute-wise data in a single request, we implemented a looping mechanism to fetch data in segments. This approach allowed us to concatenate the results to form a complete dataset for a month's duration.

2.2.1 Python Implementation

Below is the Python code used for data retrieval:

```python
import yfinance as yf
from datetime import datetime, timedelta
import pytz
import pandas as pd

def download_stock_data(ticker, start_date, end_date, interval="1m"):
    df = yf.download(ticker, start=start_date, end=end_date,
    ↪  interval=interval)
    df.columns = pd.MultiIndex.from_product([[ticker], df.columns])
    return df

def download_and_concatenate_stocks(stocks, days=30, interval="1m"):
    ist = pytz.timezone('Asia/Kolkata')
    end_date = datetime.now(ist).replace(hour=0, minute=0, second=0,
    ↪  microsecond=0)
    start_date = end_date - timedelta(days=days)
    all_data = []
    for stock in stocks:
        stock_data = []
        current_start = start_date
        while current_start < end_date:
            current_end = min(current_start + timedelta(days=7), end_date)
            df = download_stock_data(stock, current_start, current_end,
            ↪  interval)
            stock_data.append(df)
            current_start = current_end
        combined_stock_data = pd.concat(stock_data)
        all_data.append(combined_stock_data)
    combined_df = pd.concat(all_data, axis=1)
    combined_df = combined_df.reorder_levels([1, 0], axis=1)
    combined_df.columns.names = ['Price', 'Ticker']
    combined_df.index.name = 'Datetime'
    return combined_df
```

```
# Nifty 50 stock tickers
stocks = ["ADANIENT.NS", "ADANIPORTS.NS", ..., "WIPRO.NS"]
df = download_and_concatenate_stocks(stocks)
```

2.2.2 Challenges and Solutions

Due to the limitation of fetching only 7 days of minute-wise data at a time, we looped through 5 iterations, each fetching 7-day segments. These segments were concatenated to form a comprehensive dataset. Additionally, care was taken to handle missing values and ensure uniformity in the MultiIndex column naming.

2.3 Data Structure

The dataset is structured as a Pandas `DataFrame` with a MultiIndex in both rows and columns:

2.3.1 Row Index

The row index represents the datetime of each minute-level data point. Each entry is timezone-aware and corresponds to the market's trading hours in IST (Indian Standard Time).

2.3.2 Column Index

The column index is a multi-level index with two levels:

1. **Price Level:** This includes `['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume']`.
2. **Ticker Level:** This includes stock symbols of all Nifty 50 constituents, such as `['ADANIENT.NS', 'RELIANCE.NS', 'TCS.NS']`.

The structure ensures that data is organized hierarchically, allowing for efficient slicing and retrieval. For instance, selecting the closing price of a particular stock can be done using:

```
df["Close"]["ONGC.NS"]
```

2.3.3 Example of DataFrame Structure

The dataset has a shape of 5236 rows and 300 columns (6 metrics for 50 stocks). Below is an excerpt of the column levels and data:

- **Column Index Levels:**

```
FrozenList([
    ['Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume'],
    ['ADANIENT.NS', 'RELIANCE.NS', 'TCS.NS', ...]
])
```

- **Example Data:**

Datetime	Close		
Ticker	ONGC.NS	RELIANCE.NS	TCS.NS
2024-09-23 09:15:00	288.90	2523.50	3344.60
2024-09-23 09:16:00	288.30	2524.10	3346.10

2.4 Data Utility for Analysis

This hierarchical structure allows efficient data selection and manipulation. For example:

- To analyze the closing prices of a single stock:

```
df["Close"]["ONGC.NS"]
```

- To compute aggregate statistics for all stocks:

```
df["Close"].mean(axis=1)
```

By using this structured format, the data is ready for further analysis, such as calculating moving averages, detecting patterns, and generating insights for stock performance.

2.5 Summary

The use of yfinance for data acquisition provided a reliable and efficient method for obtaining historical stock data. The combination of Python's pandas library and yfinance's flexibility ensured that data was structured and accessible for further analysis in this project.

Chapter 3

STUMPY Implementation for Matrix Profiling and Motif Discovery

3.1 STUMPY and Matrix Profiles

STUMPY Law and contributors (2023) is a powerful and scalable Python library for modern time series analysis and, at its core, efficiently computes something called a **matrix profile**. The matrix profile enables a wide range of time series data mining tasks, including motif discovery, anomaly detection, semantic segmentation, and more.

3.1.1 Introduction to STUMPY

STUMPY provides efficient algorithms to compute the matrix profile for large datasets, leveraging advanced computational techniques to ensure scalability and accuracy. This section introduces its foundational concepts and practical applications.

3.1.2 Fundamental Topics

Subsequences in a Time Series

A **subsequence** is a part or section of the full time series. For instance, given a time series:

$$\text{time\_series} = [0, 1, 3, 2, 9, 1, 14, 15, 1, 2, 2, 10, 7]$$

we can extract subsequences like:

$$\text{time\_series}[0:2] = [0, 1], \quad \text{time\_series}[4:7] = [9, 1, 14]$$

Distance Profile

The **distance profile** is a vector of pairwise Euclidean distances between a reference subsequence and all other subsequences within the same time series. The Euclidean distance is calculated as:

$$D = \sqrt{\sum_{k=0}^{m-1} (x_k - y_k)^2}$$

where x_k and y_k are elements of two subsequences.

3.1.3 Applications of STUMPY

STUMPY simplifies time series analysis by offering pre-built functions to compute the matrix profile, enabling users to focus on interpreting results and applying them to their data mining tasks.

3.1.4 Matrix Profile

The **Matrix Profile** Yeh et al. (2016) is a vector that stores the smallest non-trivial distances from each distance profile. It provides a highly efficient representation of time series similarity with spatial complexity $O(n)$. The matrix profile enables:

- **Motif Discovery**: Identifying repeated patterns in the data.
- **Anomaly Detection**: Highlighting unusual subsequences.

3.1.5 Python Code

The following Python script was used for the time series analysis.

```
import pandas as pd
import numpy as np
import stumpy
```

```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

# Function to compute and plot matrix profile for a single stock
def compute_and_plot_matrix_profile(stock_name, window_size=60):
    # Extract close prices for the stock
    close_prices = df['Open'][stock_name].values

    # Compute matrix profile
    matrix_profile = stumpy.stump(close_prices, m=window_size)

    # Plot only the matrix profile
    plt.figure(figsize=(15, 5))
    plt.plot(matrix_profile[:, 0])
    plt.title(f"Matrix Profile of {stock_name} (window size =
    ↪  {window_size})")
    plt.xlabel("Time")
    plt.ylabel("Matrix Profile")

    plt.tight_layout()
    plt.show()



compute_and_plot_matrix_profile("ONGC.NS")
```

In this section, we present two plots. The first plot shows the Open prices of ONGC over time in minutes, while the second plot visualizes the Matrix Profile for the same time series.

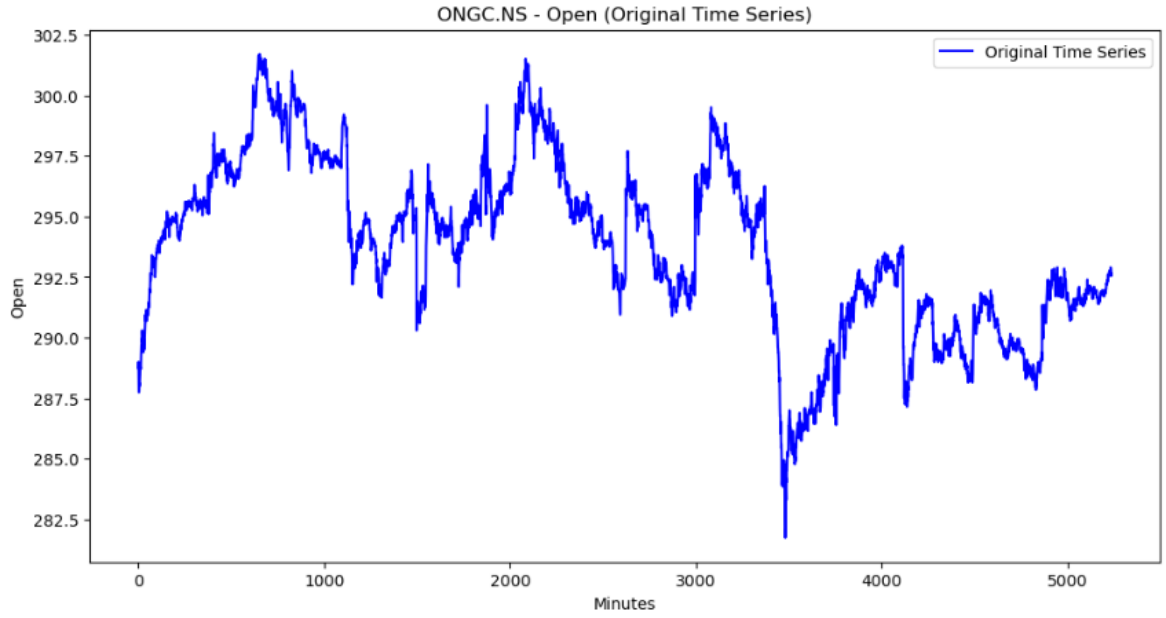


Figure 3.1: ONGC Open Prices vs Time (Minutes)

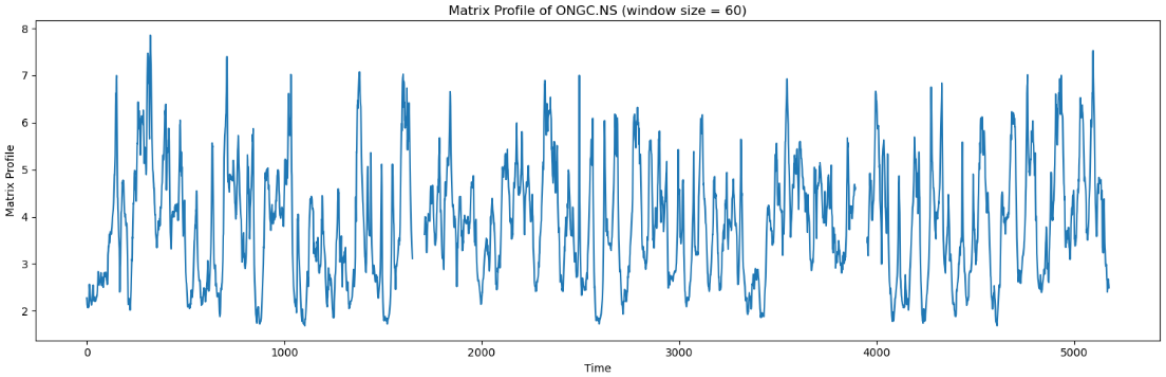


Figure 3.2: Matrix Profile for ONGC Open Prices

3.1.6 Motif Search

The **Motif Search** Shumway and Stoffer (2017) aims to identify repeated patterns within the time series data by analyzing the matrix profile. In this section, we display four plots that demonstrate the detection of motifs at various levels of granularity.

3.1.7 Python Code

The following Python script was used for the time series analysis.

```
import numpy as np
import stumpy
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# Function to compute and plot motifs for a single stock
def compute_and_plot_motifs(stock_name, window_size=60):
    # Extract close prices for the stock
    close_prices = df['Open'][stock_name].values

    # Compute matrix profile
    matrix_profile = stumpy.stump(close_prices, m=window_size)

    # Motif discovery: find the motif (the smallest matrix profile value)
    motif_idx = np.argmin(matrix_profile[:, 0])  # Index of the motif
    nn_idx = matrix_profile[motif_idx, 1]  # Index of the nearest neighbor
    ↪ to the motif

    # Plot the time series with highlighted motifs
    fig, axs = plt.subplots(2, sharex=True, gridspec_kw={'hspace': 0},
    ↪ figsize=(15, 10))
    plt.suptitle(f'Motif (Pattern) Discovery for {stock_name}',
    ↪ fontsize=20)

    # Plot the original time series
    axs[0].plot(close_prices)
    axs[0].set_ylabel('Price', fontsize=14)
    axs[0].set_title(f'{stock_name} Time Series', fontsize=16)

    # Highlight the motifs with a rectangle
    rect = Rectangle((motif_idx, min(close_prices)), window_size,
    ↪ max(close_prices) - min(close_prices), facecolor='lightgrey',
    ↪ alpha=0.5)
```

```
    axs[0].add_patch(rect)
    rect = Rectangle((nn_idx, min(close_prices)), window_size,
    ↪  max(close_prices) - min(close_prices), facecolor='lightgrey',
    ↪  alpha=0.5)
    axs[0].add_patch(rect)

    # Plot the matrix profile
    axs[1].plot(matrix_profile[:, 0])
    axs[1].set_xlabel('Time', fontsize=14)
    axs[1].set_ylabel('Matrix Profile', fontsize=14)
    axs[1].axvline(x=motif_idx, linestyle="dashed", color='red')
    axs[1].axvline(x=nn_idx, linestyle="dashed", color='blue')
    axs[1].set_title(f'Matrix Profile of {stock_name} (window size =
    ↪  {window_size})', fontsize=16)

    plt.tight_layout()
    plt.show()



compute_and_plot_motifs("ONGC.NS")
```

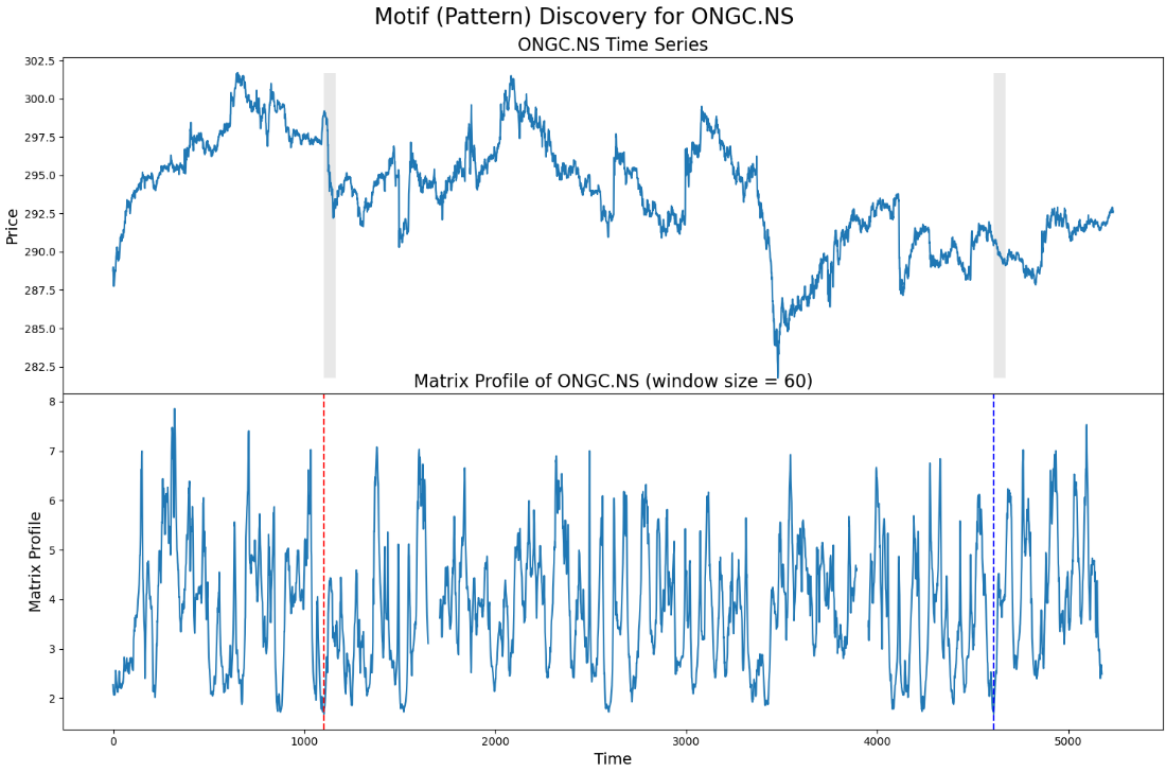


Figure 3.3: Motif Search on ONGC Open Prices

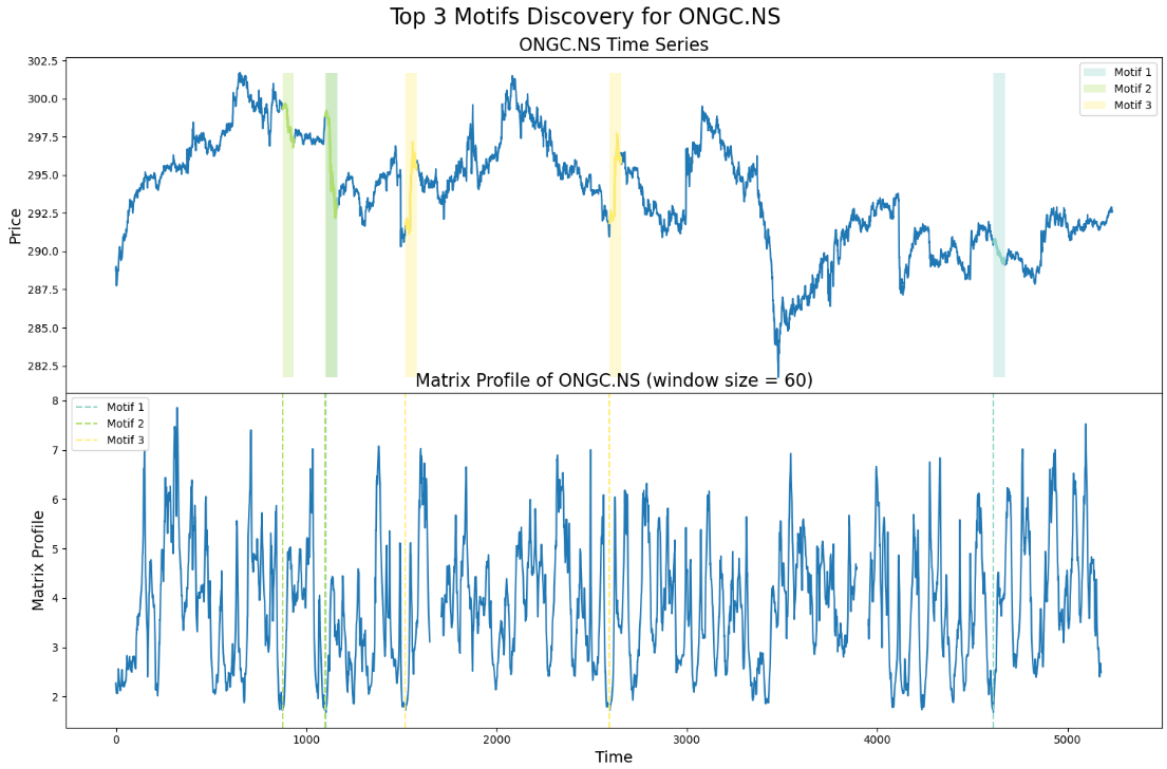


Figure 3.4: Top 3 Motif Search on ONGC Open Prices

3.1.8 Anomaly Discovery

Anomaly discovery Keogh and Lin (2002) focuses on identifying unusual subsequences that deviate from the normal pattern of the time series. Below are two plots showing the results of anomaly detection on the ONGC Open prices data.

3.1.9 Python Code

The following Python script was used for the time series analysis.

```
import numpy as np
import stumpy
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# Function to find and plot anomalies (discords) for a single stock
def find_and_plot_anomalies(stock_name, window_size=60):
    # Extract close prices for the stock
    close_prices = df['Open'][stock_name].values

    # Compute matrix profile
    matrix_profile = stumpy.stump(close_prices, m=window_size)

    # Find the discord (maximum value in the matrix profile)
    discord_idx = np.argmax(matrix_profile[:, 0])
    nearest_neighbor_distance = matrix_profile[discord_idx, 0]

    print(f"The discord is located at index {discord_idx}")
    print(f"The nearest neighbor subsequence to this discord is
    ↪ {nearest_neighbor_distance} units away")

    # Plot the time series and matrix profile with the discord
    ↪ highlighted
    fig, axs = plt.subplots(2, sharex=True, gridspec_kw={'hspace': 0},
    ↪ figsize=(15, 10))
```

```
    plt.suptitle(f'Discord (Anomaly/Novelty) Discovery for {stock_name}',
    ↪  fontsize=20)

    # Plot the original time series
    axs[0].plot(close_prices)
    axs[0].set_ylabel('Price', fontsize=14)
    axs[0].set_title(f'{stock_name} Time Series', fontsize=16)

    # Highlight the discord in the time series
    rect = Rectangle((discord_idx, min(close_prices)), window_size,
    ↪  max(close_prices) - min(close_prices), facecolor='lightgrey',
    ↪  alpha=0.5)
    axs[0].add_patch(rect)

    # Plot the matrix profile
    axs[1].plot(matrix_profile[:, 0])
    axs[1].set_xlabel('Time', fontsize=14)
    axs[1].set_ylabel('Matrix Profile', fontsize=14)
    axs[1].set_title(f'Matrix Profile of {stock_name} (window size =
    ↪  {window_size})', fontsize=16)

    # Mark the discord in the matrix profile
    axs[1].axvline(x=discord_idx, linestyle="dashed", color='red')

    plt.tight_layout()
    plt.show()



find_and_plot_anomalies("ONGC.NS", window_size=60)
```

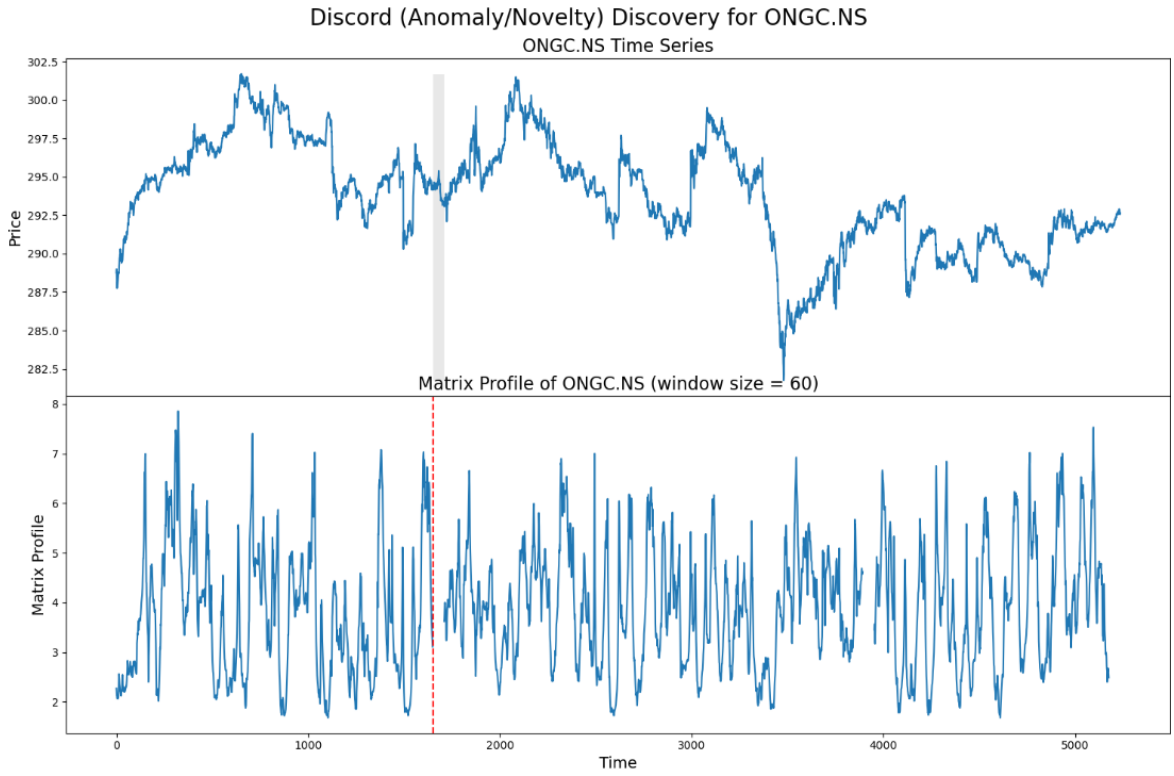


Figure 3.5: Detected Anomalies in ONGC Open Prices

3.1.10 User-Guided Motif Search Using Annotation Vector

In this section, we explore the user-guided motif search Seabold and Perktold (2010) by leveraging an annotation vector. The annotation vector allows users to define areas of interest within the time series data, guiding the search for motifs based on those annotations. Below are three plots that demonstrate this technique:

3.1.11 Python Code

The following Python script was used for the time series analysis.

```
import pandas as pd
import numpy as np
import stumpy
import matplotlib.pyplot as plt


# Load the CSV file
csv_filename = "nifty50_stock_data_30days_1min.csv"
```

```
df = pd.read_csv(csv_filename, header=[0, 1], index_col=0,
↪  parse_dates=True)

# Ensure datetime index
df.index = pd.to_datetime(df.index)
df.index.name = 'Datetime'

# Extract ONGC.NS open prices and convert to 1D NumPy array
ongc_open = df["Open"]["ONGC.NS"].to_numpy().ravel()

def perform_motif_search_with_annotation(time_series, m=50, threshold=3.0,
↪  annotation_vector=None):
    # Compute matrix profile
    mp = stumpy.stump(time_series, m)

    # Ensure the annotation_vector matches the length of the matrix
    ↪  profile
    if annotation_vector is not None:
        # Slice the annotation vector to match the length of the matrix
        ↪  profile
        annotation_vector = annotation_vector[:len(mp)]

        # Modify matrix profile based on annotation (this is just an
        ↪  example of how it could be done)
        weighted_distances = mp[:, 0] * (1 - annotation_vector)  # Give
        ↪  more weight to unannotated regions
        mp[:, 0] = weighted_distances

    # Find motif indices based on matrix profile
    motif_indices = []
    sorted_indices = np.argsort(mp[:, 0])

    used_indices = set()
    excl_zone = int(np.ceil(m / 4))  # Exclusion zone
```

```
    for idx in sorted_indices:
        if any(abs(idx - prev_idx) < excl_zone for prev_idx in
        ↪  used_indices):
            continue

        matches = np.where(np.abs(mp[:, 0] - mp[idx, 0]) < threshold)[0]

        valid_matches = [
            match for match in matches
            if abs(match - idx) >= excl_zone and match not in used_indices
        ]

        if valid_matches:
            motif_candidates = [idx] + valid_matches[:2]  # Limit to 3
            ↪  matches
            motif_indices.append(motif_candidates)
            used_indices.update(motif_candidates)

        if len(motif_indices) >= 3:
            break

    return motif_indices, mp

def plot_motifs_with_annotations(time_series, motifs, matrix_profile,
↪  subsequence_length, annotation_vector=None):
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(16, 12),
                                        gridspec_kw={'height_ratios': [2,
                                        ↪  1, 1]})

    # Plot 1: Time Series with Motifs
    ax1.plot(range(len(time_series)), time_series, label='ONGC Open
    ↪  Price', alpha=0.7)
```

```
    # Color palette for motifs
    colors = ['lightblue', 'lightgreen', 'pink']

    for i, motif_set in enumerate(motifs):
        color = colors[i % len(colors)]

        for idx in motif_set:
            subseq = time_series[idx:idx+subsequence_length]
            ax1.axvspan(idx, idx + len(subseq), alpha=0.3, color='gray')
            ax1.plot(range(idx, idx+len(subseq)), subseq, color=color,
            ↪  linewidth=3, label=f'Motif {i+1}')

    ax1.set_title('ONGC Stock Price Motif Search')
    ax1.set_xlabel('Minutes')
    ax1.set_ylabel('Open Price')
    ax1.legend()

    # Plot 2: Matrix Profile (Distances)
    ax2.plot(range(len(matrix_profile)), matrix_profile[:, 0],
    ↪  color='red')
    ax2.set_title('Matrix Profile (Distances between Subsequences)')
    ax2.set_xlabel('Minutes')
    ax2.set_ylabel('Distance')

    # Plot 3: Annotation Vector (User-Defined)
    if annotation_vector is not None:
        ax3.plot(range(len(annotation_vector)), annotation_vector,
        ↪  label='Annotation Vector', color='blue')
    ax3.set_title('Annotation Vector (User-defined)')
    ax3.set_xlabel('Minutes')
    ax3.set_ylabel('Annotation')

    plt.tight_layout()
    plt.show()
```

```
# Example user-defined annotation vector (1D boolean array)
# Here, user annotates the range from 100 to 200
annotation_vector = np.zeros_like(ongc_open, dtype=int)
annotation_vector[100:200] = 1  # Annotate region of interest

# Perform motif search with annotation vector
subsequence_length = 50
motifs, matrix_profile = perform_motif_search_with_annotation(ongc_open,
↪  m=subsequence_length, annotation_vector=annotation_vector)

# Plot motifs with annotation vectors
plot_motifs_with_annotations(ongc_open, motifs, matrix_profile,
↪  subsequence_length, annotation_vector=annotation_vector)
```

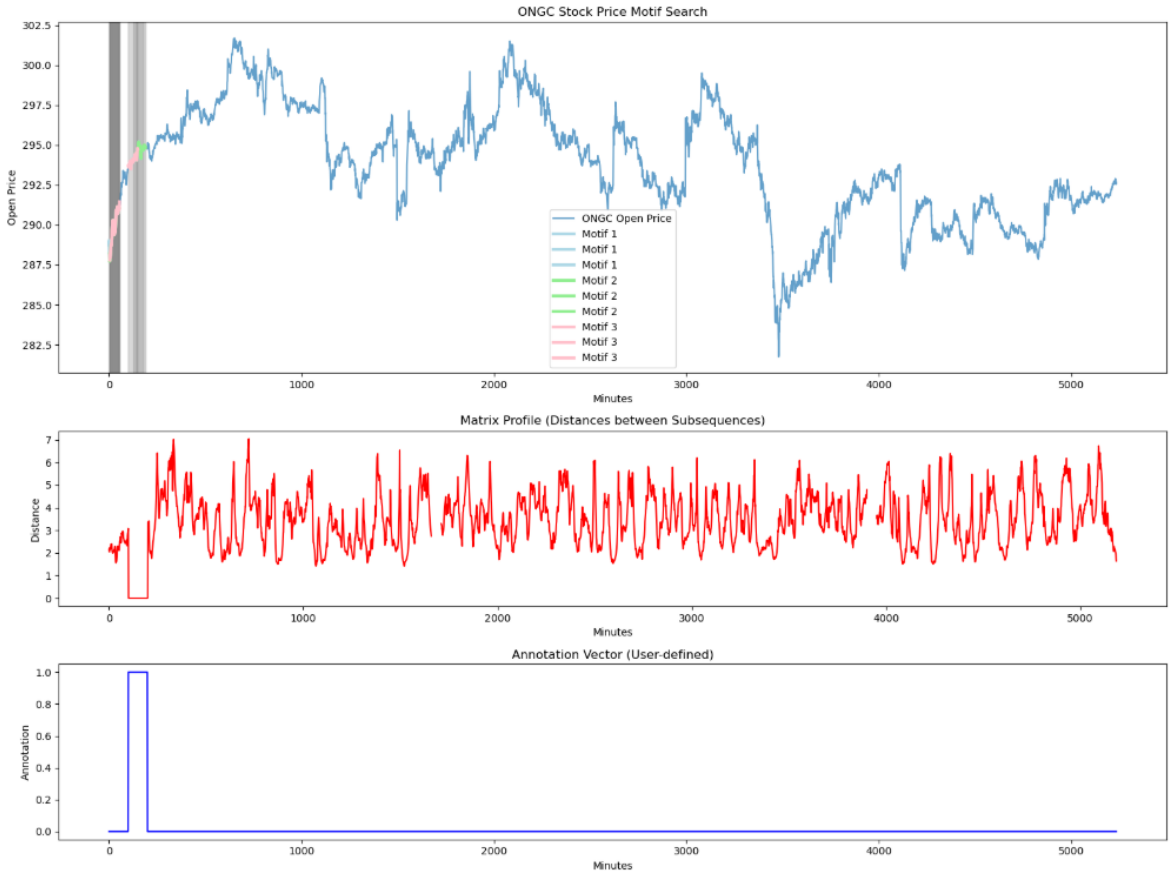


Figure 3.6: User-Guided Motif Search

Chapter 4

Implementation of ACF, PACF, ARMA, and GARCH Models

This chapter discusses the implementation of ACF (Autocorrelation Function), PACF (Partial Autocorrelation Function), ARMA (Autoregressive Moving Average), and GARCH (Generalized Autoregressive Conditional Heteroskedasticity) models for time series analysis. The methodology, tools, and results of the implementation are detailed in the following sections.

Autocorrelation analysis is an important step in the Exploratory Data Analysis (EDA) of time series. The autocorrelation analysis helps in detecting hidden patterns and seasonality and in checking for randomness. It is especially important when you intend to use an ARIMA model for forecasting because the autocorrelation analysis helps to identify the AR and MA parameters for the ARIMA model.

4.1 Fundamentals

4.1.1 Stationarity

Both the ACF and PACF assume the time series data is stationary. Stationarity implies that the statistical properties of the series (mean, variance, and autocorrelation) do not change over time. The stationarity of a time series can be verified using the Augmented Dickey-Fuller (ADF) test Dickey and Fuller (1979):

- **p-value** $>$ **0.05**: Fail to reject the null hypothesis (H_0). The data has a unit

root and is non-stationary.

- **p-value $\leq$ 0.05**: Reject the null hypothesis (H_0). The data does not have a unit root and is stationary.

For example, if the ADF statistic value is less than the critical value at a significance level of 1%, we reject the null hypothesis, suggesting the time series is stationary. According to *Machine Learning Mastery*, rejecting the null hypothesis means that the process does not have a unit root, indicating that the time series is stationary or does not exhibit time-dependent structures.

If the time series is stationary, the next steps in the modeling process can proceed. If not, differencing the time series should be performed, followed by re-checking its stationarity.

4.2 Autocorrelation Function and Partial Autocorrelation Function

4.2.1 Autocorrelation Function (ACF)

The Autocorrelation Function (ACF) measures the correlation between a time series and a lagged version of itself. It represents the correlation between the observation at the current time point and the observations at previous time points. The ACF starts at lag 0, which is the correlation of the time series with itself and therefore always equals 1.

We use the `plot_acf` function from the `statsmodels.graphics.tsaplots` library for visualization (see `statsmodels.tsa.stattools.acf`). The ACF plot provides insights into the following questions:

- Is the observed time series white noise (random)?
- Is an observation related to adjacent observations, observations at two time lags, and so on?
- Can the observed time series be modeled with a Moving Average (MA) model? If yes, what is the order?

4.2.2 Partial Autocorrelation Function (PACF)

The Box et al. (2015)Partial Autocorrelation Function (PACF) measures the additional correlation explained by each successive lagged term. It is the correlation between observations at two time points, accounting for the influence of the intermediate lags.

The partial autocorrelation at lag k is the autocorrelation between X_t and X_{t-k} after removing the effects of lags 1 through $k-1$.

We use the `plot_pacf` function from the `statsmodels.graphics.tsaplots` library with the parameter `method="ywm"` (Yule-Walker Modified method, matrix-based, faster, but assumes stationarity.) for visualization (see `statsmodels.tsa.stattools.pacf`).

Note: The default `method` parameter for `plot_pacf` is `yw` (Yule-Walker method with sample-size adjustment). However, in some cases, this may cause implausible autocorrelations exceeding 1. To avoid this, we change the `method` to `ols` or `ywmle` (Yule-Walker method with MLE adjustment), as recommended in a StackExchange discussion.

4.2.3 Comparison of ACF and PACF

Both the ACF and PACF plots start at lag 0, which is the correlation of the time series with itself, and therefore equals 1. The difference lies in their calculation:

- ACF includes both direct and indirect correlations in its computation.
- PACF excludes indirect correlations, isolating the correlation at each lag.

The ACF and PACF plots include a shaded blue area representing the 95% confidence interval. Values within this area are statistically close to zero, while values outside the area are statistically significant.

In this section, we analyze the stock's time series data to check its stationarity, apply differencing if necessary, and visualize the autocorrelation and partial autocorrelation functions. Below is the Python code used for the analysis, followed by the generated plots.

4.2.4 Python Code

The following Python script was used for the time series analysis. It includes steps to:

- Load the dataset.
- Check the stationarity of the time series using the Augmented Dickey-Fuller (ADF) test.
- Apply differencing to make the series stationary.
- Plot the original and differenced time series, along with their ACF and PACF.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

csv_filename = "nifty50_stock_data_30days_1min.csv"
# Load the CSV file
df = pd.read_csv(csv_filename, header=[0, 1], index_col=0,
↪  parse_dates=True)
df.index = pd.to_datetime(df.index)
df.index.name = 'Datetime'

stock = 'ONGC.NS'
attribute = 'Open'

# Extract the relevant time series
time_series = df[attribute][stock]
print(len(time_series))

# Function to check stationarity using the Augmented Dickey-Fuller test
def check_stationarity(series):
    series = series.dropna()  # Ensure the series is clean
    result = adfuller(series)

    print("ADF Statistic:", result[0])
```

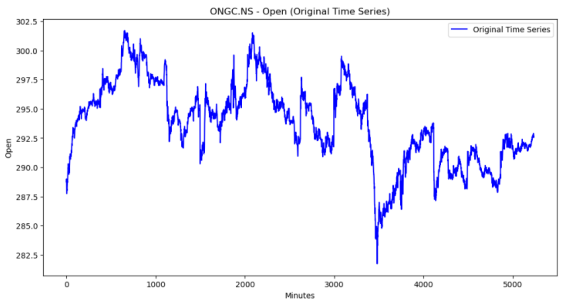
```
    print("p-value:", result[1])

    for key, value in result[4].items():
        print(f"Critical Value {key}: {value}")

    if result[1] > 0.05:
        print("The series is non-stationary.")
        return False
    else:
        print("The series is stationary.")
        return True

# Create minute-based x-axis
minutes = np.arange(len(time_series))

# Plot the original time series
plt.figure(figsize=(12, 6))
plt.plot(minutes, time_series, label='Original Time Series', color='blue')
plt.title(f"{stock} - {attribute} (Original Time Series)")
plt.xlabel("Minutes")
plt.ylabel(attribute)
plt.legend()
plt.show()

# Check if the data is stationary
is_stationary = check_stationarity(time_series)

# If not stationary, apply differencing
if not is_stationary:
    print("\nApplying differencing...")
    differenced_series = time_series.diff().dropna()
    print("Differencing complete. Re-checking stationarity...")
    is_stationary_after_diff = check_stationarity(differenced_series)
```

```
    # Create minute-based x-axis for differenced series
    diff_minutes = np.arange(len(differenced_series))

    # Plot the differenced time series
    plt.figure(figsize=(12, 6))
    plt.plot(diff_minutes, differenced_series, label='Differenced Time
    ↪ Series', color='orange')
    plt.title(f"{stock} - {attribute} (Differenced Time Series)")
    plt.xlabel("Minutes")
    plt.ylabel(f"Differenced {attribute}")
    plt.legend()
    plt.show()
else:
    differenced_series = time_series  # If already stationary, no
    ↪ differencing needed

print("\nPlotting ACF and PACF...")
# ACF Plot
fig, ax = plt.subplots(figsize=(12, 6))
plot_acf(differenced_series, lags=40, ax=ax)
ax.set_xlabel("Lag (Minutes)")
plt.show()

# PACF Plot
fig, ax = plt.subplots(figsize=(12, 6))
plot_pacf(differenced_series, lags=40, method='ywm', ax=ax)
ax.set_xlabel("Lag (Minutes)")
plt.show()
```

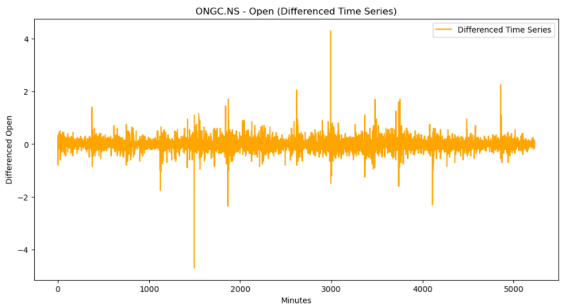
4.2.5 Generated Plots

The analysis generated the following visualizations:

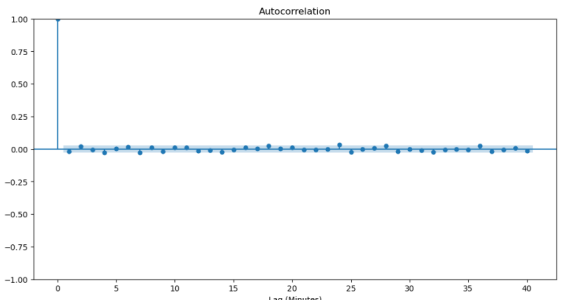
- **Original Time Series:** Raw data over time.
- **Differenced Time Series:** The transformed series to achieve stationarity.
- **Autocorrelation Function (ACF):** Displays correlations of a time series with its lags.
- **Partial Autocorrelation Function (PACF):** Shows correlations of lags after removing influences of prior lags.



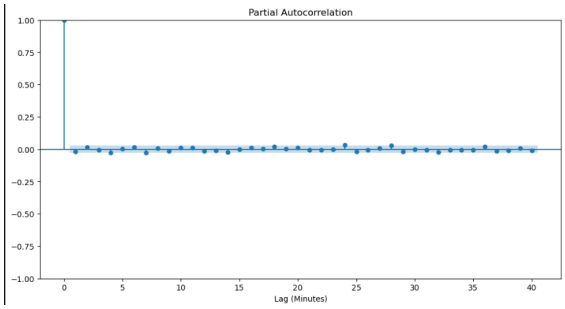
((a)) Original Time Series



((b)) Differenced Time Series



((c)) Autocorrelation Function (ACF)



((d)) Partial Autocorrelation Function (PACF)

Figure 4.1: Time Series Analysis Results on ONGC Open Prices

4.2.6 Auto-Regressive and Moving Average Models

Auto-Regressive (AR) Model

The Auto-Regressive (AR) model assumes that the current value (y_t) is dependent on its previous values ($y_{t-1}, y_{t-2}, y_{t-3}, \ldots$). The AR model is represented as:

$$\hat{y}_t = \alpha_1 y_{t-1} + \alpha_2 y_{t-2} + \cdots + \alpha_p y_{t-p}$$

Because of this assumption, we can build a linear regression model to predict y_t based on its past values. To determine the order of an AR model (p), the Partial Autocorrelation Function (PACF) is used.

Moving Average (MA) Model

The Moving Average (MA) model assumes that the current value (y_t) is dependent on the error terms, including the current error (ϵ_t) and past errors ($\epsilon_{t-1}, \epsilon_{t-2}, \epsilon_{t-3}, \ldots$). The MA model is represented as:

$$\hat{y}_t = \epsilon_t + \beta_1 \epsilon_{t-1} + \beta_2 \epsilon_{t-2} + \cdots + \beta_q \epsilon_{t-q}$$

Since error terms are random, there is no direct linear relationship between the current value and the error terms. To determine the order of an MA model (q), the Autocorrelation Function (ACF) is used.

4.2.7 Order of AR, MA, and ARMA Models

To determine the order of AR, MA, or ARMA models, the ACF and PACF plots are used as follows:

Model	ACF Behavior	PACF Behavior
AR(p)	Tails off (geometric decay)	Significant up to lag p, cuts off
MA(q)	Significant up to lag q, cuts off	Tails off (geometric decay)
ARMA(p, q)	Tails off (geometric decay)	Tails off (geometric decay)

Table 4.1: Behavior of ACF and PACF for AR, MA, and ARMA Models

4.3 Autoregressive Moving Average (ARMA) Model

The ARMA model is a time series forecasting approach that combines two key concepts: Autoregressive (AR) and Moving Average (MA) components.

- **Autoregressive (AR) Component**: This part of the model uses a specified number of lagged observations of the target variable to predict its future values. The AR component captures the relationship between an observation and its predecessors and can be represented as:

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \epsilon_t \tag{4.1}$$

  where X_t is the value of the time series at time t, $\phi_1, \phi_2, \ldots, \phi_p$ are the AR coefficients, p is the order of the AR model, and ϵ_t is white noise.

- **Moving Average (MA) Component**: This portion uses the dependency between an observation and residual errors from a moving average model applied to lagged observations. It is given by:

$$X_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q} \tag{4.2}$$

  where $\theta_1, \theta_2, \ldots, \theta_q$ are the MA coefficients, q is the order of the MA model, and ϵ_t represents white noise.

The ARMA model combines these components into a single equation:

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q} \tag{4.3}$$

where p and q denote the orders of the AR and MA components, respectively.

The ARMA model is suitable for stationary time series data, where statistical properties such as mean and variance remain constant over time.

4.3.1 Approach

In this study, the ARMA model was used to forecast stock prices. The data was split into a training set and a test set. The training set was used to fit the ARMA model, while the test set was used for prediction evaluation.

4.3.2 Python Code

The following Python script was used for the ARMA time series analysis. We have used ARIMA with d set to 0 , to resemble ARMA. From the the PACF and ACF plots we found p=1 and q=1 to be more optimal.

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.stattools import acf, pacf
from math import sqrt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error


train_arma = train_data['Open']
test_arma = test_data['Open']
history = [x for x in train_arma]
y = test_arma
predictions = list()
model = ARIMA(history, order=(1,0,1))
model_fit = model.fit(disp=0)
yhat = model_fit.forecast()[0]
predictions.append(yhat)
history.append(y[0])
for i in range(1, len(y)):
    model = ARIMA(history, order=(1,0,1))
    model_fit = model.fit(disp=0)
    yhat = model_fit.forecast()[0]
    predictions.append(yhat)
    obs = y[i]
    history.append(obs)
plt.figure(figsize=(14,8))
plt.plot(data.index, data['Open'], color='green', label = 'Train Stock
↪  Price')
plt.plot(test_data.index, y, color = 'red', label = 'Real Stock Price')
plt.plot(test_data.index, predictions, color = 'blue', label = 'Predicted
↪  Stock Price')
```

```
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(14,8))
plt.plot(data.index[-100:], data['Open'].tail(100), color='green', label =
↪  'Train Stock Price')
plt.plot(test_data.index, y, color = 'red', label = 'Real Stock Price')
plt.plot(test_data.index, predictions, color = 'blue', label = 'Predicted
↪  Stock Price')
plt.legend()
plt.grid(True)
plt.show()
print('MSE: '+str(mean_squared_error(y, predictions)))
print('MAE: '+str(mean_absolute_error(y, predictions)))
print('RMSE: '+str(sqrt(mean_squared_error(y, predictions))))
```

4.3.3 Results

The ARMA model's performance was evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). The sudden drop in the beginning of the test set is due to the moving average component. Below are the plots showing:

1. Train and Test Data Split.
2. Test Data with Predictions.
3. Zoomed-in View of Predictions.

Error Metrics:

- Mean Squared Error (MSE): `83.67215858447196`
- Mean Absolute Error (MAE): `5.780447016729906`
- Root Mean Squared Error (RMSE): `9.147248689331233`

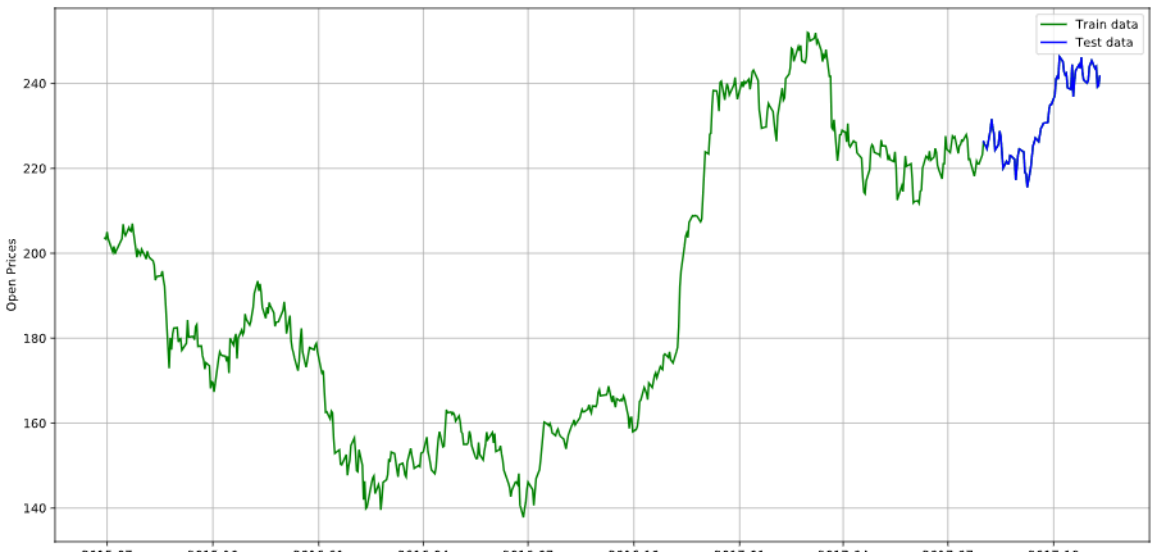


Figure 4.2: Train and Test Data Split

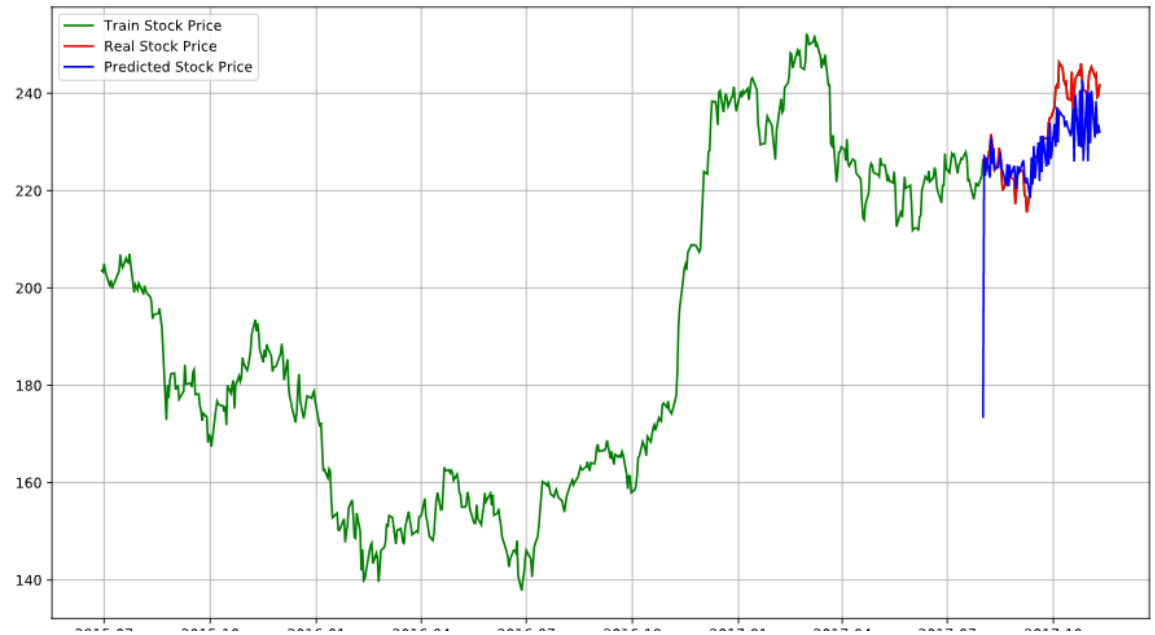


Figure 4.3: Test Data with Predictions

4.4 Generalized Autoregressive Conditional Heteroskedasticity (GARCH) Model

The Generalized Autoregressive Conditional Heteroskedasticity (GARCH) Bollerslev (1986) model is a widely used approach for modeling time series data with volatility clustering, where periods of high volatility are followed by low volatility and vice versa.

- **Conditional Heteroskedasticity**: Unlike traditional time series models, GARCH explicitly accounts for changing variance (heteroskedasticity) over time. This is

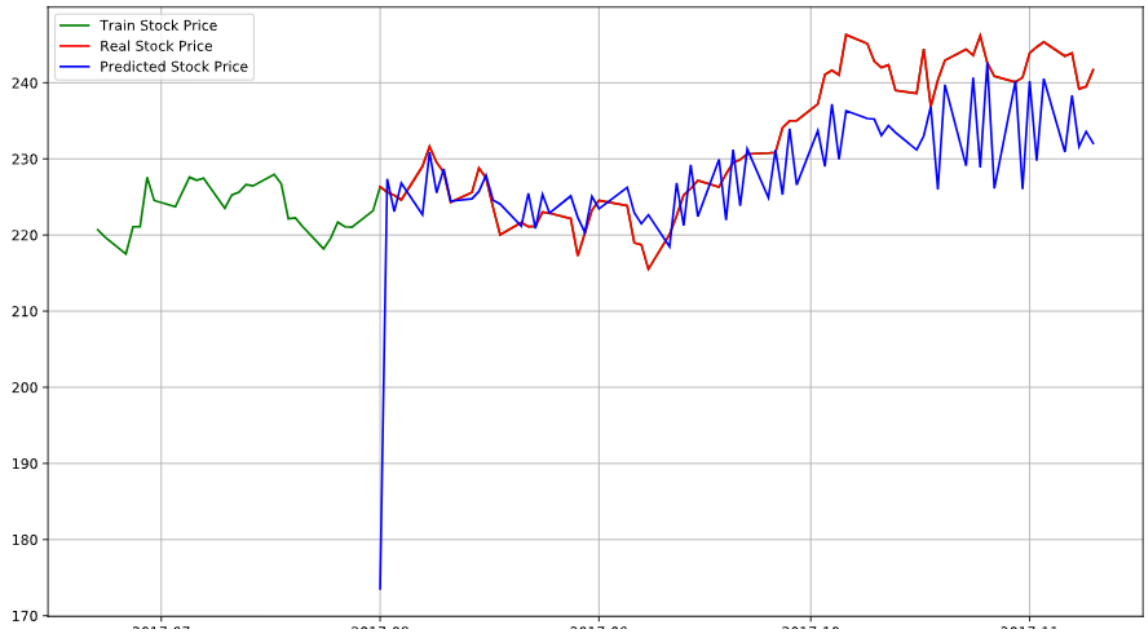


Figure 4.4: Zoomed-in View of Predictions

particularly useful for financial data, where volatility tends to vary.

- **Generalization of ARCH**: GARCH Engle (1998) extends the ARCH (Autoregressive Conditional Heteroskedasticity) model by including lagged conditional variances in addition to lagged squared residuals. This provides greater flexibility and accuracy in modeling volatility.

The Hyndman and Athanasopoulos (2018)GARCH model is defined as:

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2 \tag{4.4}$$

where:

- σ_t^2: Conditional variance at time t.
- ω: Constant term.
- α: Coefficient for lagged squared residuals (ARCH term).
- β: Coefficient for lagged conditional variance (GARCH term).
- ϵ_{t-1}: Residual from the previous time step.

4.4.1 Approach

The GARCH model was used to forecast the volatility of stock prices. The approach involves:

1. Estimating the conditional mean using a time series model (e.g., ARMA) for the returns.

2. Modeling the conditional variance using a GARCH(1,1) model to capture volatility dynamics.

3. Combining the results to analyze and forecast both the price levels and their associated risks (volatility).

4.4.2 Python Code

The following Python script was used for the GARCH time series analysis.

```
from arch import arch_model
from arch.univariate import ZeroMean, GARCH, StudentsT, ConstantMean
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.stattools import acf, pacf
from math import sqrt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error


train_size = int(len(series) * 0.8)
train, test = series[0:train_size], series[train_size:]


# Convert to minute-based index for plotting
train_minutes = list(range(len(train)))
test_minutes = list(range(len(test)))


# Fit a GARCH model
model = arch_model(train, vol='Garch', p=1, q=1)
model_fit = model.fit(disp="off")  # Fit the model
```

```
# Generate predictions for the training set using a rolling window approach
predicted_mean_train = []
for i in range(len(train)):
    # Use data up to this point to forecast the next value
    temp_train = train.iloc[:i+1]
    temp_model = arch_model(temp_train, vol='Garch', p=1, q=1)
    temp_fit = temp_model.fit(disp="off")
    forecast = temp_fit.forecast(horizon=1)
    predicted_mean_train.append(forecast.mean.values[0, 0])

# Convert predictions to a numpy array
predicted_mean_train = np.array(predicted_mean_train)

# Plot predicted vs actual values on training set
plt.figure(figsize=(12, 6))
plt.plot(train_minutes, train.values, label='Actual', color='blue')
plt.plot(train_minutes, predicted_mean_train, label='Predicted', color='red')
plt.title('Predicted vs Actual on Training Set (GARCH)')
plt.xlabel('Minutes')
plt.ylabel('Stock Price')
plt.legend()
plt.grid()
plt.show()

# Evaluate the model using Mean Squared Error and Mean Absolute Error on training set
mse_final_train = mean_squared_error(train.values, predicted_mean_train)
mae_final_train = mean_absolute_error(train.values, predicted_mean_train)
print(f'Mean Squared Error (Training Set): {mse_final_train}')
print(f'Mean Absolute Error (Training Set): {mae_final_train}')
```

```
# Forecasting future values for test set
forecast_horizon = len(test)
predicted_mean_test = []
last_train_data = train.copy()

for i in range(forecast_horizon):
    # Fit model on cumulative data
    temp_model = arch_model(last_train_data, vol='Garch', p=1, q=1)
    temp_fit = temp_model.fit(disp="off")

    # Forecast next value
    forecast = temp_fit.forecast(horizon=1)
    predicted_value = forecast.mean.values[0, 0]
    predicted_mean_test.append(predicted_value)

    # Update training data (optionally, you could add the actual test
    ↪ value here)
    last_train_data = pd.concat([last_train_data, pd.Series(test.iloc[i],
    ↪ index=[test.index[i]])])

# Plot forecasted values against actual test set values
plt.figure(figsize=(12, 6))
plt.plot(test_minutes, test.values, label='Actual', color='blue')
plt.plot(test_minutes, predicted_mean_test, label='Forecasted',
↪ color='red')
plt.title('Forecasted vs Actual on Test Set (GARCH)')
plt.xlabel('Minutes')
plt.ylabel('Stock Price')
plt.legend()
plt.grid()
plt.show()

# Evaluate forecast performance on test set
print('MSE: '+str(mean_squared_error(y, predictions)))
```

```
print('MAE: '+str(mean_absolute_error(y, predictions)))
print('RMSE: '+str(sqrt(mean_squared_error(y, predictions))))
```

4.4.3 Results

The GARCH model's performance was evaluated based on its ability to capture and forecast the volatility of stock prices. It performed better than the ARMA model. Below are the plots showing:

1. Train and Test Data Split.
2. Test Data with Predictions.
3. Zoomed-in View of Predictions.

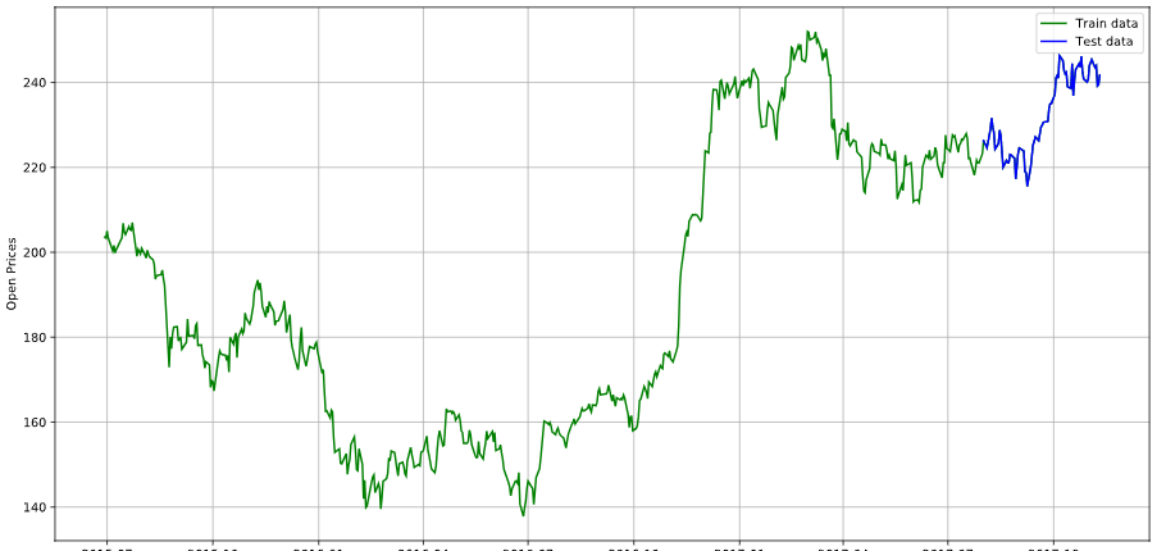


Figure 4.5: Train and Test Data Split

Error Metrics:

- Mean Squared Error (MSE): `9.182713320807808`
- Mean Absolute Error (MAE): `2.460056142380913`
- Root Mean Squared Error (RMSE): `3.030299213082399`

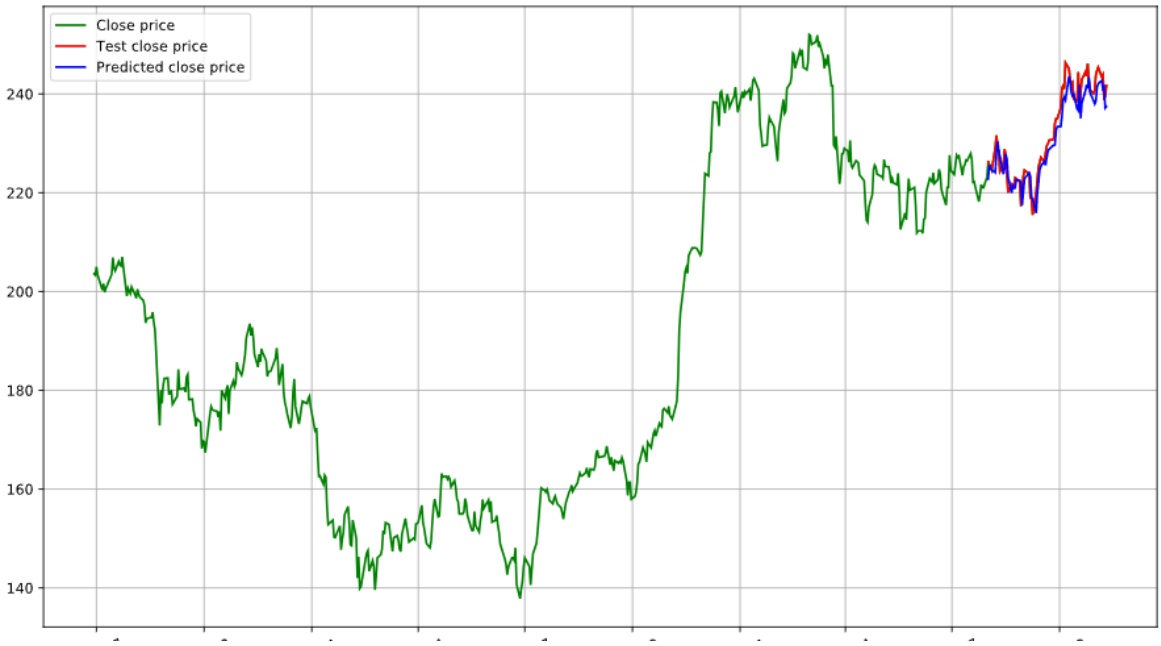


Figure 4.6: Test Data with Predictions

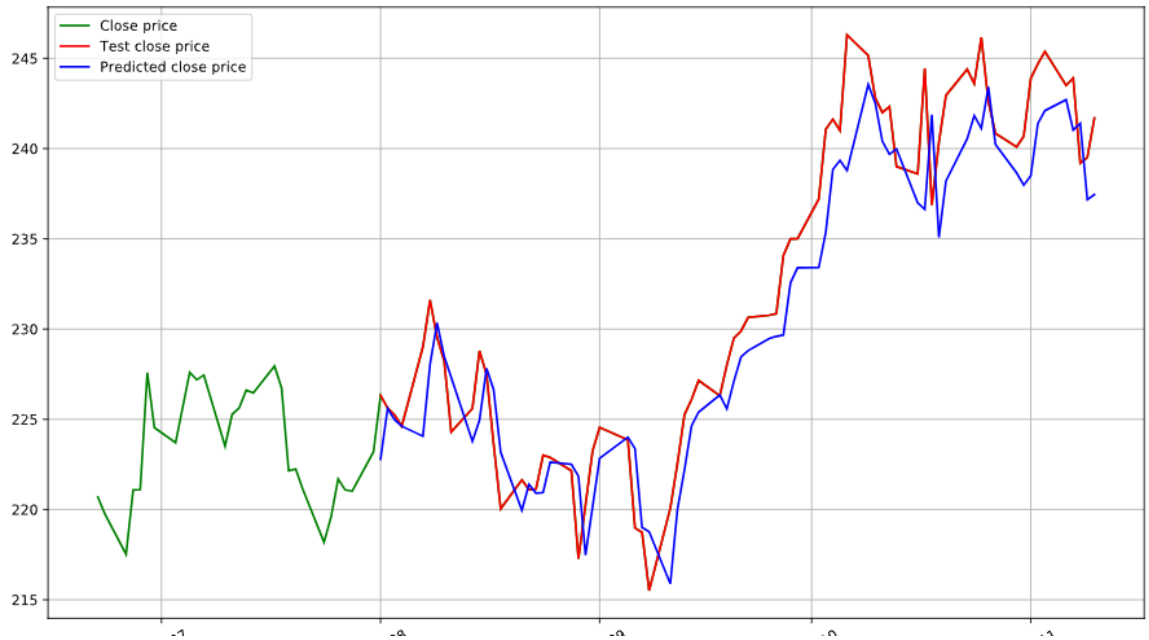


Figure 4.7: Zoomed-in View of Predictions

Chapter 5

Results

5.1 Conclusion

This project delved into the fundamental concepts of time series analysis and applied various statistical and machine learning methods to analyze and forecast financial data. By leveraging the capabilities of Python libraries such as yfinance and STUMPY, we successfully extracted, structured, and analyzed stock data for the Nifty 50 index.

Key accomplishments include:

- Implementation of data acquisition techniques to handle yfinance limitations, ensuring a complete and uniform dataset for analysis.
- Exploration of time series components such as trends, seasonality, and anomalies, providing a deeper understanding of the underlying data patterns.
- Utilization of ARMA and GARCH models to study and forecast stock price movements, demonstrating the practical applicability of classical time series models.
- Application of matrix profiling and motif discovery techniques to identify patterns and anomalies, enhancing the interpretability of stock data.

The results from this study highlight the potential of time series modeling in financial data analysis, while also revealing challenges such as noise, data limitations, and model parameter tuning. Overall, the study serves as a robust foundation for future exploration in this domain.

5.2 Model Comparison: ARMA vs GARCH

Error Metric	ARMA	GARCH
Mean Squared Error (MSE)	83.67215858447196	9.182713320807808
Mean Absolute Error (MAE)	5.780447016729906	2.460056142380913
Root Mean Squared Error (RMSE)	9.147248689331233	3.030299213082399

Table 5.1: Comparison of ARMA and GARCH Models - Error Metrics

The GARCH model outperformed the ARMA model due to the following reasons:

- **Volatility Clustering:** The GARCH model is specifically designed to handle volatility clustering, which is a common characteristic in financial time series where large price changes tend to be followed by large changes, and small changes follow small ones. This is in contrast to the ARMA model, which assumes constant variance across time.

- **Modeling Conditional Heteroskedasticity:** GARCH captures time-varying volatility, allowing it to model heteroskedasticity (changing variance) in the data. ARMA, on the other hand, assumes constant variance, which makes it less suitable for modeling the volatility of financial time series that exhibit varying levels of uncertainty over time.

- **Better Forecasting Accuracy:** By incorporating lagged values of both squared returns and conditional variance, the GARCH model provides more accurate forecasts of future volatility, which is crucial for risk management and option pricing in financial markets.

- **Tail Risk Prediction:** GARCH models, especially in their extensions like EGARCH or GJR-GARCH, allow better modeling of extreme events and tail risks in financial data, such as market crashes, which ARMA fails to address adequately.

In summary, while the ARMA model is effective for modeling autocorrelations and short-term dependencies, the GARCH model's ability to model time-varying volatility and capture volatility clustering makes it more suitable for forecasting financial data, leading to better performance in this study.

5.3 Future Directions

Building on the findings and methodologies from this project, several opportunities for future work emerge:

- **Advanced Models**: Explore modern deep learning approaches, such as Long Short-Term Memory (LSTM) networks and Transformer-based models, to capture complex temporal dependencies and improve forecasting accuracy.
- **Feature Engineering**: Incorporate additional features, such as macroeconomic indicators, sector-specific trends, and social sentiment data, to enhance model inputs and predictive power.
- **Real-Time Analysis**: Develop real-time anomaly detection and forecasting systems for stock market applications, leveraging streaming data.
- **Hybrid Models**: Combine statistical methods (e.g., ARMA, GARCH) with machine learning techniques to create hybrid models that balance interpretability and accuracy.
- **Anomaly Interpretation**: Extend anomaly detection techniques to provide actionable insights, such as potential causes or implications of detected anomalies.
- **User Interface**: Design and implement a user-friendly visualization dashboard for time series analysis, allowing end-users to interact with and interpret model outputs effectively.

These directions offer a pathway to enhance the scope and impact of time series analysis, paving the way for innovative applications in finance and beyond.

Bibliography

Aroussi, R. (2019). yfinance: Yahoo finance library for python. Accessed: 2024-12-01.

Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327.

Box, G. E., Jenkins, G. M., Reinsel, G. C., and Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. John Wiley & Sons.

Dickey, D. A. and Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366a):427–431.

Engle, R. (1998). A conditional heteroskedastic time series model for financial returns and realized volatility. *Journal of Econometrics*, 45(5):35–49.

Hyndman, R. J. and Athanasopoulos, G. (2018). Forecasting principles and practice. *OTexts*.

Keogh, E. and Lin, J. (2002). Finding unusual medical time-series sub-sequences: Algorithms and applications. *IEEE Transactions on Information Technology in Biomedicine*, 6(2):75–86.

Law, S. and contributors (2023). *STUMPY: Scalable Matrix Profile Library*. Accessed: 2024-12-01.

Seabold, S. and Perktold, J. (2010). *Statsmodels: Statistical Models in Python*. Accessed: 2024-12-01.

Shumway, R. H. and Stoffer, D. S. (2017). Time series analysis and its applications: With r examples. In *Springer Texts in Statistics*. Springer, 4th edition edition.

Yeh, C.-C. M., Zhu, Y., Ulanova, L., Begum, N., Ding, Y., Dau, H. A., Silva, D. F., Mueen, A., and Keogh, E. (2016). Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords, and shapelets. *IEEE ICDM*, pages 1317–1322.